

METODOLOGÍA I

Trabajo de Integración: del Discovery al primer Change, con Spec-Driven Development y Active Stack

OBJETIVO GENERAL

Recorrer, de principio a fin y sobre un producto real, el flujo de trabajo que vieron en los videos de **SDD con OpenSpec** y de **Active Stack**: partir de una idea sin ningún código, investigarla con método (Discovery), fundar el proyecto con un agente de IA (base de conocimiento, roadmap y reglas del agente) y llevar **una funcionalidad** desde su especificación hasta su archivado mediante un ciclo OPSX completo.

El producto sobre el que van a trabajar es un **sistema de gestión de turnos y agenda de pacientes para consultorios odontológicos, pensado para el mercado argentino**. No van a programar “lo que se les ocurra”: van a decidir qué construir a partir de evidencia verificable, y a demostrar que cada decisión quedó registrada y es trazable.

PUNTO DE PARTIDA

Este trabajo da por sabido lo explicado en los videos. Si algún concepto de la siguiente tabla no les resulta claro, vuelvan a verlos antes de empezar:

- **Video SDD + OpenSpec**: qué es una especificación, escenarios verificables y el ciclo `explore` → `propose` → `apply` → `archive`.
- **Active Stack (partes 1, 2 y 3)**: harnesses, modos de instalación (Lite/Full/Custom), `/active-orchestrator`, las siete etapas de la fundación, el protocolo de inspección (Continuar / Ajustar / Parar acá), la bitácora compartida, el ciclo OPSX y Engram.

MARCO TEÓRICO

Concepto	Aplicación en este trabajo
Discovery	Convierte “tengo una idea” en “sé qué voy a construir y por qué”, antes de decidir arquitectura. Acá lo van a resolver con una investigación de mercado exhaustiva y verificable.
Harness	Andamiaje que rodea al modelo (skills, configuración, herramientas externas). Active Stack instala ese andamiaje sobre su proyecto.
Fundación del proyecto	Las siete etapas de <code>/active-orchestrator</code> : OpenSpec Init, Discovery, KB Creator, Roadmap Generator, Find Skill, Skill Registry y Agent Instruction.

Concepto	Aplicación en este trabajo
Knowledge-base y <code>CHANGES.md</code>	Mapa de contexto del proyecto y plan de implementación. De ellos sale la lista de changes de la cual eligen el que van a implementar.
Protocolo de inspección	Al final de cada etapa deciden entre Continuar, Ajustar o Parar acá. Lo que se evalúa no es que hayan avanzado, sino que hayan <i>leído</i> lo generado y corregido lo que estaba mal.
Ciclo OPSX	Explore, propose, apply, verify y archive sobre un cambio concreto. La especificación aprobada en propose es el contrato de la implementación.
Engram	Memoria persistente del agente entre sesiones. Van a mostrar qué quedó guardado y cómo se recupera.

MODALIDAD Y ORGANIZACIÓN

- **Individual o en grupo de hasta 3 integrantes.** Todos los integrantes deben participar en alguna parte del flujo.
- En grupo, cada integrante debe ser **responsable de al menos una etapa** de la fundación o del ciclo OPSX (por ejemplo: uno lleva Discovery, otro KB + Roadmap, otro el change). Ser responsable significa conducir esa etapa con el agente, revisar lo generado y decidir Continuar/Ajustar.
- La participación de cada integrante tiene que ser **demostrable**: aparece explicada en la reflexión escrita, se ve en el historial de commits del repositorio (cada persona con sus propios commits) y cada uno habla en el video de defensa.
- Un único repositorio y una única entrega por individuo o por grupo.

CASO PRÁCTICO

El trabajo se divide en ocho etapas consecutivas. Las etapas 1 a 5 corresponden a la fundación del proyecto (las siete etapas de `/active-orchestrator` agrupadas de forma didáctica); la etapa 6 es el ciclo OPSX y la 7 cierra con evidencia y memoria. **No avancen a una etapa sin haber revisado la anterior**: en cada una usen el protocolo de inspección y anoten qué decidieron y por qué.

Etapla 0. Preparación del entorno

- **Herramientas previas:** Go, Git y Node instalados, y un agente de código (OpenCode o Claude Code) funcionando.
- **Active Stack en modo Full:** clonar `github.com/Group-Active-IA/active-stack`, ejecutar `build.bat` (Windows) o `build.sh` (Linux/macOS) y correr el instalador eligiendo el modo **Full**, tal como se muestra en la Parte 1.
- **Repositorio nuevo y vacío** para el trabajo (Git inicializado, sin código previo). Nombre sugerido: `turnos-odontologia`.
- **Evidencia:** una captura del instalador finalizado y de la lista de harnesses instalados.

Etapa 1. Iniciar la fundación (`/active-orchestrator` y `openspec init`)

- Ejecutar `/active-orchestrator` dentro del repositorio vacío. Debe correr primero `openspec init` , que deja lista la estructura base de OpenSpec.
- Cuando aparezca la pregunta de la bifurcación (“¿hacemos estudio de mercado?”), responder **Sí**: se decide una sola vez y en este trabajo el Discovery es obligatorio.
- **Evidencia:** el archivo de estado `.active-orchestrator-state.json` con la etapa completada (se incluye en el repositorio).

Etapa 2. Discovery: investigación de mercado

Es el corazón del trabajo y la etapa donde más tiempo conviene invertir. Se ejecuta con la skill `discovery-research` , que responde un checklist de once puntos: problema, usuarios, casos de uso, competidores, funcionalidades necesarias, funcionalidades opcionales, reglas de negocio, integraciones, restricciones, riesgos y preguntas abiertas. Si le pasan URLs de competidores, la skill puede delegar en `web-scraper` para investigarlas antes de preguntarles.

Además de esa ronda de preguntas, deben encargarle a su agente la siguiente investigación, **tal cual está redactada** (pueden copiarla y pegarla como consigna):

Consigna de investigación para el agente

Actuá como analista sénior de mercado y producto SaaS especializado en salud digital y odontología en Argentina y Latinoamérica.

Objetivo: realizar una búsqueda exhaustiva de sistemas odontológicos que permitan gestionar turnos y agenda de pacientes. El propósito es identificar soluciones existentes, comparar sus funcionalidades y detectar oportunidades para desarrollar un producto competitivo.

Alcance geográfico y prioridad: Argentina; Latinoamérica; soluciones internacionales relevantes que puedan utilizarse o inspirar un producto para el mercado argentino.

Buscá exclusivamente fuentes verificables: sitios oficiales de cada proveedor, documentación pública, páginas de precios, centros de ayuda, videos oficiales, marketplaces confiables y reseñas verificadas. Indicá la URL y la fecha de consulta de cada fuente. No inventes funcionalidades: si una característica no está demostrada públicamente, consignéla como “No evidenciado”.

Para cada sistema identificado, relevá:

- Nombre del producto, empresa proveedora, país de origen y URL oficial.
- Segmento objetivo: odontólogo independiente, consultorio, clínica, cadena, centro multidisciplinario o laboratorio.
- Modalidad: SaaS, nube, aplicación móvil, instalación local o híbrida.
- Gestión de agenda: vista diaria/semanal, múltiples profesionales, múltiples sillones o boxes, duración variable de prestaciones, bloqueos, sobretornos y prevención de solapamientos.
- Turnos digitales: reserva online 24/7, enlace de reserva, web, redes sociales, chatbot, WhatsApp, confirmación, cancelación, reprogramación y lista de espera.
- Automatización: recordatorios, confirmaciones, seguimiento de ausentes, recuperación de pacientes, controles periódicos y campañas.
- Gestión odontológica: ficha clínica, anamnesis, odontograma, periodontograma, imágenes/radiografías, presupuestos y planes de tratamiento.
- Administración: caja, cobros, facturación, señas, Mercado Pago, obras sociales/prepagas, liquidaciones, reportes y métricas.
- Integraciones: WhatsApp Business API, Google Calendar, sitio web, APIs públicas, Mercado Pago, AFIP/ARCA, radiología, firma digital y herramientas de marketing.
- Operación: sucursales, roles y permisos, auditoría, exportación de datos, soporte y capacitación.
- Seguridad y cumplimiento: datos sensibles de salud, privacidad, trazabilidad, respaldo, control de acceso y adecuación declarada a la normativa argentina aplicable.
- Modelo comercial: prueba gratuita, plan gratuito, precios publicados, moneda, modalidad de cobro y costos adicionales visibles.
- Fortalezas, limitaciones y diferenciales evidentes.
- Evidencia de adopción: casos de éxito, cantidad de clientes solo si está publicada y reseñas verificables.

Presentá el resultado en cuatro secciones:

A. Tabla comparativa de al menos 15 sistemas, ordenados por relevancia para Argentina.

B. Matriz de puntuación de 0 a 5, con estos pesos: Gestión de turnos y automatización, 25 %; Funcionalidad odontológica clínica, 20 %; Integraciones locales y WhatsApp, 15 %; Administración, cobros y facturación, 15 %; Experiencia del paciente, 10 %; Seguridad, exportación y trazabilidad, 10 %; Precio y facilidad de adopción, 5 %.

C. Análisis competitivo: funcionalidades que ya son estándar de mercado; funcionalidades diferenciadoras reales; vacíos frecuentes del mercado argentino; oportunidades de innovación para un nuevo sistema de gestión odontológica.

D. Recomendación final: cinco competidores prioritarios para analizar mediante demo; tres productos de referencia para experiencia de usuario; un MVP sugerido con funcionalidades imprescindibles, diferenciadores y funcionalidades a dejar para etapas posteriores.

Redactá en español formal, con criterio crítico. Diferenciá siempre entre una funcionalidad comprobada y una afirmación comercial del proveedor.

Un agente de IA no deja huecos sin resolver: si no encuentra un dato, tiende a inventarlo. Por eso la verificación de fuentes es parte de la consigna y se evalúa:

- Como mínimo, **abran y comprueben personalmente 5 de las fuentes citadas** (elijan una de cada tipo: sitio oficial, página de precios, centro de ayuda, etc.). Para cada una, anoten si lo que el informe afirma realmente está en esa URL.
- Toda celda de la tabla sin respaldo público debe decir **“No evidenciado”**, no quedar vacía ni completarse por deducción.
- Si encuentran una afirmación falsa o no respaldada, corrijanla con **Ajustar** y dejen registrado el hallazgo (se usa en la reflexión).

- **Entregable de la etapa:** el informe de Discovery (docs/discovery/informe-discovery.md y su versión en PDF) con las secciones A, B, C y D, más la sección final “Verificación de fuentes”. El resultado estructurado de discovery-research (checklist de once puntos) se guarda en la misma carpeta.
- **Guardar el informe dentro del repositorio** es importante: es el insumo que la etapa siguiente va a leer.

Etapla 3. Base de conocimiento (kb-creator)

- Ejecutar la etapa. Al existir documentación fuente en docs/ , la skill puede trabajar en modo silencioso; si les hace preguntas (modo interactivo), respóndanlas con criterio y en base a lo investigado, no con respuestas al azar.
- Resultado esperado: la carpeta knowledge-base/ con los diez archivos canónicos: visión, descripción, actores, modelos de datos, reglas de negocio, features, flujos, arquitectura, decisiones y preguntas abiertas.
- **Revisión obligatoria (Ajustar si corresponde):** verificar que la knowledge-base refleje el informe de Discovery (competidores, MVP recomendado, vacíos del mercado) y no un producto genérico. Como mínimo, comprobar que las reglas de negocio incluyan las de agenda (duración de prestaciones, sillones o boxes, prevención de solapamientos) y que “preguntas abiertas” contenga los puntos que el Discovery dejó como “No evidenciado”.

Etapa 4. Roadmap (`roadmap-generator`)

- Generar `CHANGES.md` a partir de la knowledge-base: dependencias entre tareas, asignación con contexto suficiente, camino crítico y riesgos.
- **Revisión obligatoria:** el roadmap debe estar alineado con el MVP recomendado en la sección D del informe. Si incluye funcionalidades que el propio Discovery dejó para etapas posteriores, o si omite una imprescindible, usen Ajustar.

Etapa 5. Skills y reglas del agente (`find-skill`, `skill-registry`, `agent-instruction`)

- **find-skill:** solo recomienda. Elijan qué skills instalar (o ninguna) y **justifiquen la decisión** en una línea por cada recomendación.
- **skill-registry:** debe quedar generado `.atl/skill-registry.md`.
- **agent-instruction:** es la única etapa realmente interactiva. Revisen cada regla candidata antes de confirmarla; deben quedar `AGENTS.md` y `CLAUDE.md` en la raíz. Agreguen a mano, si corresponde, al menos **una regla propia del dominio** (por ejemplo, “nunca almacenar datos reales de pacientes en el repositorio ni en los ejemplos”).

Etapa 6. Ciclo OPSX sobre un change (`explore` → `propose` → `apply` → `verify` → `archive`)

Con el proyecto fundado, elijan **un único change** de `CHANGES.md` y llévenlo por el ciclo completo.

6.1 Cómo elegir el change

- Tiene que pertenecer al **MVP** que ustedes mismos recomendaron en la sección D.
- Tiene que tener **reglas de negocio verificables**. Ejemplo recomendado: “Crear un turno evitando solapamientos por profesional y por sillón/box”. Otros válidos: “Reprogramar un turno” o “Cancelar un turno y liberar el horario”.
- Tiene que ser lo bastante chico como para terminarse: una sola funcionalidad, no un módulo.
- La elección se justifica por escrito (dos o tres líneas) en el `proposal.md`.

6.2 Las cinco fases

- **Explore:** relevar el problema y el estado del código antes de proponer nada. Guardar el resultado o un resumen del relevamiento.
- **Propose:** generar los artefactos del change (`proposal.md`, `design.md`, `specs/` y `tasks.md`). Los escenarios de la especificación tienen que estar redactados como comportamiento verificable (dado / cuando / entonces) y cubrir **al menos 3 escenarios**: uno feliz, uno de error de negocio y uno de borde (por ejemplo, un turno que empieza justo cuando termina otro). **No aprueben a la primera:** lean el spec completo y usen *Ajustar* si el alcance quedó de más, de menos o inventado.
- **Apply:** implementar contra la especificación aprobada, sin decisiones de diseño improvisadas. Si algo no estaba en el spec, no se inventa en Apply: se vuelve a Propose. El stack tecnológico lo definen ustedes, según lo que hayan dejado en la knowledge-base; la implementación puede ser una API o un módulo con su lógica de dominio, no hace falta interfaz gráfica.

- **Verify:** correr los tests y los health checks. Debe haber tests automatizados que cubran cada escenario del spec. No se pasa a Archive con algo en rojo. Si su instalación no ofrece una fase de verificación como comando propio, ejecuten la suite de tests a mano y documenten el resultado.
- **Archive:** cerrar el ciclo. El change debe quedar archivado y el estado del proyecto actualizado.

Etapa 7. Memoria del agente (Engram)

- Mostrar qué quedó guardado en Engram sobre este proyecto (decisiones, hallazgos, convenciones).
- Hacer **una consulta real** que recupere contexto de una sesión anterior y explicar cómo eso evita que el agente arranque de cero. Esta evidencia se muestra en el video.

ENTREGABLES Y CÓMO DEBE VERSE EL RESULTADO

1. Repositorio (link o .zip)

Con esta estructura mínima (los nombres exactos de algunos archivos pueden variar según la versión instalada de cada skill; lo importante es que estén todos los elementos):

```
turnos-odontologia/
├── AGENTS.md
├── CLAUDE.md
├── CHANGES.md
├── .active-orchestrator-state.json
├── .atl/
│   └── skill-registry.md
├── docs/
│   └── discovery/
│       ├── informe-discovery.md
│       └── informe-discovery.pdf
├── knowledge-base/           (los 10 archivos canónicos)
├── openspec/
│   ├── specs/                (specs vigentes)
│   └── changes/
│       └── archive/
│           └── AAAA-MM-DD-crear-turno-sin-solapamientos/
│               ├── proposal.md
│               ├── design.md
│               ├── tasks.md
│               └── specs/
├── src/                      (implementación del change)
├── tests/                    (tests de cada escenario)
└── README.md                 (cómo instalar y correr los tests)
```

- El repositorio debe poder clonarse y ejecutar los tests siguiendo únicamente el `README.md`.
- El **historial de commits** debe mostrar el proceso (fundación, discovery, change), no un único commit final. En grupo, cada integrante tiene commits propios.
- **Sin secretos:** ninguna clave de API ni credencial en el repositorio.

2. Informe de Discovery (PDF)

Documento profesional con este orden:

1. **Portada** (título, materia, integrantes, fecha).
2. **Resumen ejecutivo** (máximo una página): qué se investigó, principales hallazgos y qué MVP se recomienda.
3. **A. Tabla comparativa** de al menos 15 sistemas, ordenada por relevancia para Argentina. Cada fila con nombre, proveedor, país, URL oficial y fecha de consulta. Las celdas sin evidencia pública dicen “No evidenciado”.
Ejemplo del formato esperado de cada fila:

Producto / Proveedor	País · Segmento	Reserva online	Recordatorios	Fuente y fecha
[Producto ejemplo] [Empresa]	[País] · Consultorio	Comprobado (enlace público de reserva)	No evidenciado	[URL del centro de ayuda] · consultada el AAAA-MM-DD

4. **B. Matriz de puntuación** de 0 a 5 con los pesos indicados y el total ponderado por sistema. Debe incluir una nota metodológica breve: qué significó cada nota y cómo se trataron los datos “No evidenciados”.
5. **C. Análisis competitivo** en cuatro subsecciones (estándar de mercado, diferenciadores reales, vacíos del mercado argentino, oportunidades de innovación).
6. **D. Recomendación final** con los cinco competidores para demo, los tres productos de referencia de experiencia de usuario y el MVP dividido en tres listas: imprescindible, diferenciador y etapas posteriores.
7. **Verificación de fuentes:** las 5 (o más) fuentes que comprobaron ustedes, qué afirmaba el informe, qué encontraron y qué corrigieron.
8. **Referencias** completas.

En todo el informe se distingue entre “**funcionalidad comprobada**” (la muestra el producto, la documentación o un video oficial) y “**afirmación comercial**” (lo dice el proveedor sin demostrarlo). Recomendación: un informe claro y bien fundado vale más que uno largo.

3. Video de defensa (8 a 12 minutos)

- **Todos los integrantes aparecen y hablan.** Cámara encendida al menos en la presentación de cada uno.
- Contenido y orden sugeridos: (1) qué producto eligieron y qué concluyó el Discovery, con el MVP recomendado; (2) recorrido por el repositorio: knowledge-base, roadmap y AGENTS.md ; (3) el change elegido, mostrando **el momento real en que revisaron Propose** y qué corrigieron con Ajustar; (4) Apply/Verify: los tests pasando; (5) Archive y estado final; (6) consulta a Engram; (7) conclusiones.
- Se muestra la pantalla trabajando de verdad, no solo diapositivas.
- Cada integrante explica la etapa de la que fue responsable.

4. Reflexión escrita (1 a 2 páginas)

Documento breve, redactado por ustedes (no generado por el agente), que responda:

- ¿Qué información del Discovery resultó incorrecta, no verificable o inventada por el agente? ¿Cómo lo detectaron?
- ¿En qué etapa usaron *Ajustar*, qué estaba mal y qué habría pasado si hubieran apretado Continuar?
- ¿Qué diferencia notaron entre implementar con una especificación aprobada y pedir código con un prompt suelto?
- ¿Qué parte de la fundación (knowledge-base, roadmap, reglas) le aportó más valor al ciclo OPSX y cuál menos?
- En grupo: qué hizo cada integrante y en qué etapa.

CRITERIOS DE EVALUACIÓN

Criterio	Qué se observa	Puntos
Informe de Discovery	Compleitud de A-D, 15 sistemas como mínimo, fuentes con URL y fecha, uso correcto de “No evidenciado”, distinción entre comprobado y comercial, criterio crítico en C y D.	25
Verificación de fuentes	Comprobaron personalmente al menos 5 fuentes y corrigieron lo que no estaba respaldado.	10
Fundación del proyecto	Knowledge-base coherente con el Discovery, roadmap alineado con el MVP, AGENTS.md / CLAUDE.md con reglas revisadas, registro de skills generado.	20
Change OPSX	Elección justificada, spec con al menos 3 escenarios verificables, implementación fiel al spec, tests por escenario en verde, change archivado.	25
Video de defensa	Claridad, se ve el proceso real, participación de todos los integrantes, evidencia de Engram.	10
Reflexión escrita	Honestidad y profundidad; evidencia de uso real del protocolo de inspección.	5
Trazabilidad y participación	Historial de commits con el proceso, README reproducible, sin secretos, aporte individual demostrable.	5
Total		100

CONSIDERACIONES

- ✔ **Uso responsable de la IA:** el agente es una herramienta de trabajo, no el autor. Todo lo que se entregue tiene que haber sido leído, entendido y validado por ustedes.
- ✔ **Verificar antes de confiar:** las URLs, precios y funcionalidades pueden cambiar o estar mal citados. La fecha de consulta es obligatoria.

- ✓ **No aprobar en automático:** el tiempo que se dedica a ajustar en las etapas iniciales se ahorra después, cuando implementan.
- ✓ **Datos de pacientes:** no usen datos reales de personas. Todos los ejemplos, pruebas y capturas deben usar datos ficticios.
- ✓ **Independencia de los productos analizados:** se analizan con fines educativos; no se contactan proveedores ni se crean cuentas con datos personales reales para evaluarlos.
- ✓ **Autoría:** el trabajo debe ser propio. El uso de IA se declara y se explica en la reflexión.
- ✓ **Alcance realista:** es mejor un solo change bien especificado, probado y archivado que varios a medias.

CONCLUSIONES ESPERADAS

- Poder conducir un proyecto desde una idea hasta una funcionalidad terminada, con cada decisión respaldada por evidencia y registrada en archivos versionables.
- Distinguir entre información verificada e información plausible generada por un agente, y saber cómo comprobarla.
- Usar el protocolo de inspección (Continuar / Ajustar / Parar acá) con criterio propio, entendiendo que corregir temprano es más barato.
- Experimentar en primera persona que una especificación aprobada funciona como contrato de implementación y permite verificar el resultado con tests.
- Reconocer el rol de la memoria persistente y de las reglas del agente (`AGENTS.md`) para sostener el trabajo entre sesiones.